

JavaScript Trainer Notes

Organized version — **original code kept unchanged**

The material below is arranged topic-by-topic for easier study. The code blocks are reproduced as provided.

1. Variables, Function and If–Else

```
let a=201;//global variable

function variables(){//arguments
//10
let num=prompt("enter a number")
if(num==1){
    alert("sunday")
}
else if(num==2){
    alert("monday")
}
else if(num==3){
    alert("tuesday")
}
else if(num==4){
    alert("wednesday")
}
else if(num==5){
    alert("thursday")
}
else if(num==6){
    alert("friday")
}
else if(num==7){
    alert("saturday")
}

else{
    alert("invalid input")
}
}
```

Topic: Global variable, function, prompt(), alert(), if, else-if and else.

2. Arrays and Array Methods

Trainer's complete HTML/JavaScript code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</script>
/*ARRAYS
Arrays are ordered collections of data. They can hold multiple values under
a single variable name, and the values can be of different data types.
*/
//TRAVERSAL OF ARRAYS
//using for loop
// let arr = [1, 2, 3, 4, 5];
// for(let i=0;i<=arr.length-1;i++){
//     document.writeln(arr[i]+"")
// }
// // using for each loop
// let arr1=['m','c','e','t'];
// arr1.forEach(function(data){
//     document.writeln(data);
// });

// // using for of loop
// let arr2=[11,12,13,14];
// result=""
// for(let data of arr2){
//     result+=data+1
// }
// document.writeln(result + "<br>");
// // using map method
// let arr3=[1,2,3,4,5];
// let mul=arr3.map(function(num){
//     return num*2;
```

```

// })

// document.writeln(mul)
// practice other ways lik while, do-while, for in etc..

// Methods used in arrays
//PUSH():Adds one or more elements to the end of an array.
let a=[1,3]
a.push(4)
console.log(a)

let mcet=[12,23,4,5,6]
mcet.push(7)
mcet.push('a')
mcet.push('@')
mcet.pop()
mcet.shift()
mcet.unshift(12)
let mcet1= mcet.slice(1,3)
mcet.splice(1,3,'a','b')
document.write(mcet+"<br>")

document.write(mcet1)
//POP():Removes the last element from an array and returns it.
let b=[1,2,4]
b.pop(2)
console.log(b)
//shift():Removes the first element from an array and returns it.
let c = ['a', 'b', 'c'];
let first = c.shift();
console.log(first)
//unshift(): Adds one or more elements to the beginning of an array.
let d=['b','c']
d.unshift('a')
console.log(d)
//slice():Creates a new array by extracting a portion of an existing array.
let e=[1,2,3,4,5]
let ae=e.slice(1,2)
console.log(ae)
//splice(): Adds or removes elements from a specific index.
let f=[1,2,3,4,5,6]
f.splice(1,3,'a','b')
console.log(f)
//map(): Creates a new array with the results of calling a provided function on every element.
let g = [1, 2, 3];
let map = g.map(n => n + 2);
console.log(map);
//filter(): Creates a new array with all elements that pass the test implemented by the provided function.
let h=[1, 2, 3];
let filter = g.map(n => n < 2);
console.log(filter);

// like this we have many more methods that can be researched by you self

</script>

</head>
<body>

</body>
</html>

```

3. Arrays and IS Methods — Product Assignment

Trainer's assignment and solution, organized without changing the solution code.

```
let products = [
  { id: 1, name: 'Laptop', price: 999 },
  { id: 2, name: 'Smartphone', price: 699 },
  { id: 3, name: 'Tablet', price: 499 }
];

// Add a new product
function addProduct(product) {
  products.push(product);
}

// Remove a product by ID
function removeProductById(id) {
  products = products.filter(product => product.id !== id);
}

// Update product details by ID
function updateProductById(id, updatedDetails) {
  products = products.map(product => {
    if (product.id === id) {
      return { ...product, ...updatedDetails };
    }
    return product;
  });
}

// Calculate total inventory value
function calculateTotalValue() {
  return products.reduce((acc, product) => acc + product.price, 0);
}

// Find products within a specified price range
function findProductsInRange(minPrice, maxPrice) {
  return products.filter(product => product.price >= minPrice && product.price <= maxPrice);
}

// Sort products by price
function sortProductsByPrice() {
  products.sort((a, b) => a.price - b.price);
}

// Search products by name
function searchProductsByName(searchTerm) {
  return products.filter(product => product.name.toLowerCase().includes(searchTerm.toLowerCase()));
}

// Test the functions
addProduct({ id: 4, name: 'Headphones', price: 199 });
console.log('Products after adding new product:', products);

removeProductById(2);
console.log('Products after removing ID 2:', products);

updateProductById(3, { price: 450 });
console.log('Products after updating ID 3:', products);

console.log('Total inventory value:', calculateTotalValue());

console.log('Products in the range $400 - $1000:', findProductsInRange(400, 1000));

sortProductsByPrice();
console.log('Products after sorting by price:', products);

console.log('Products containing "phone":', searchProductsByName('phone'));
```

4. Functions — Types and Examples

```
function function_name(){  
}
```

FUNCTIONS-types

1. Anonymous function
2. arrow function
3. Call Back Functions
4. Higher-Order Function
5. Pure Functions

4.1 Pure Function Assignment

```
function cube(n) {  
    return n * n * n;  
}  
  
console.log(cube(3));  
  
function doubleArray(arr) {  
    return arr.map(n => n * 2);  
}  
  
const numbers = [1, 2, 3, 4];  
const doubled = doubleArray(numbers);  
  
console.log(doubled);  
console.log(numbers);  
  
function filterAndMultiply(numbers, multiplier) {  
    return numbers  
        .filter(n => n >= 5)  
        .map(n => n * multiplier);  
}  
  
const data = [3, 5, 8, 2, 10, 1];  
const result = filterAndMultiply(data, 3);  
  
console.log(result);  
console.log(data);
```

4.2 Anonymous Functions

```
const area = (function(width, height) {  
    return width * height;  
})(10, 5);  
  
console.log(area);
```

4.3 Arrow Functions

```
const numbers = [1, 2, 3, 4];  
const squaredNumbers = numbers.map(n => n * n);  
  
console.log(squaredNumbers);  
  
const products = [  
    { name: 'Laptop', price: 999, quantity: 1 },  
    { name: 'Mouse', price: 25, quantity: 3 },  
    { name: 'Keyboard', price: 45, quantity: 2 },  
    { name: 'Monitor', price: 150, quantity: 1 },  
    { name: 'USB Cable', price: 10, quantity: 5 }  
];  
  
const expensiveProducts = products.filter(product => product.price > 20);  
const totalValues = expensiveProducts.map(product => product.price * product.quantity);  
  
console.log(expensiveProducts)  
console.log(totalValues);
```

4.4 Callback Functions

```
function processNumber(num, callback) {  
    callback(num * 2);  
}  
  
processNumber(5, function(result) {  
    console.log(result);  
});
```

```
function performOperation(num1, num2, callback) {  
    return callback(num1, num2);  
}  
  
const add = (a, b) => a + b;  
  
const result = performOperation(8, 4, add);  
console.log(result);
```

4.5 Higher-Order Functions

```
function filterNumbers(arr, callback) {  
    return arr.filter(callback);  
}  
  
function isEven(num) {  
    return num % 2 === 0;  
}  
  
const numbers = [1, 2, 3, 4, 5, 6];  
const evenNumbers = filterNumbers(numbers, isEven);  
  
console.log(evenNumbers);  
  
function createMultiplier(factor) {  
    return function(number) {  
        return number * factor;  
    };  
}  
  
const double = createMultiplier(2);  
  
const numbers = [1, 2, 3, 4, 5];  
const doubledNumbers = numbers.map(double);  
  
console.log(doubledNumbers);
```

5. Functions — Complete HTML Example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
<button onclick="trial()">Click me</button>
</body>
  <script>
/*PURE FUNCTION
def: when ever we have a requirement of producing same output given the same input
without side effects that means it doesn't modify the external state such as
global variables, input parameters etc..
*/
//example
function function_name (){
  return ;
}

/*ANONYMOUS FUNCTION

def: these functions are declared without a name , we can use this anonymous functions
in all the places that we are using the normal functions.
we use this functions mainly in callback functions and event listners.
*/
//example fro writing the simple anonymous function
var add=(function(a,b){
  c=a+b;
  return c;

})(5,10)

function trial(){
document.write(add+1)
}

/*ARROW FUNCTIONS
def: These functions are usefull for writing single expression functions.
This function is stored in variable in th ememory, easy to access.

*/
// Example: Arrow function to add two numbers
const add = (a, b) => a + b;

console.log(add(2, 3));
// Example: Arrow function with single parameter
const square = n => n * n;

console.log(square(4));

// Example: Arrow function with multiple lines
const greet = name => {
  const message = `Hello, ${name}!`;
  return message;
};
console.log(greet('mcet'));
/*CALLBACK FUNCTION
A callback function is a function passed into another function as an
argument and is executed after some operation is completed. This mechanism
allows you to run custom code once a particular task or event has finished.
*/

// Example: Using a callback function to handle a delayed operation
function fetchData(callback) {
  setTimeout(() => {
    const data = "Data fetched successfully!";
    callback(data);
  }, 2000);
}

fetchData(function(result) {
  console.log(result);
});
/*A higher-order function is a function that either:

Takes one or more functions as arguments.
Returns a function as its result.
Data Transformation: When you need to transform data in a collection using
```

```
    functions like map, filter, and reduce.
*/
// Example: Higher-order function that takes a function as an argument
function applyFunction(arr, func) {
    return arr.map(func);
}

// Function to be used as an argument
function square(x) {
    return x * x;
}

const numbers = [1, 2, 3, 4];
const squaredNumbers = applyFunction(numbers, square);

console.log(squaredNumbers); // Outputs: [1, 4, 9, 16]

</script>
</html>
```

6. DOM — Document Object Model

Complete trainer code, organized as one section.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <script>
/*
DOM:Document object model
By using dom java script can access, change or remove any elements of HTML document and also can
create new elements at any position.

When a webpage is loaded, browser create DOM of webpage
*/

</script>

<div>
  <form>
    <h1>MCET</h1>
  </form>
</div>
<!--
Document object has properties, methods
Using this document object properties and methods we can
1.select html elements
2.modify html elements
3.remove/delete html elements
4.create html elements
5.Add/remove/Change styles to html elements
-->
<!--
Methods to select HTML Elements:
1.document.getElementById(): returns elements with the specific id
-->

<h1 id="mcetbyid">hello</h1>
<script>
  var v1=document.getElementById("mcetbyid")
  console.log(v1)
</script>

<!--
2.document.getElementsByClassName(): returns list of all elements belongs to the specified class
-->
<div class="mcetbyclass">

  <h1>element1</h1>
  <p>element2<p>
  <h3>element3</h3>
</div>
<div class="mcetbyclass">

  <h1>element1</h1>
  <p>element2<p>
  <h3>element3</h3>
</div>
<div class="mcetbyclass">

  <h1>element1</h1>
  <p>element2<p>
  <h3>element3</h3>
</div>
<script>
  var v=document.getElementsByClassName("mcetbyclass")
  console.log(v)
</script>
<!--
3.document.getElementsByTagName(): Returns list of all elements with the specified tag
-->
<div class="mcetbyclass">

  <h1>element1</h1>
  <p>element2<p>
  <h3>element3</h3>
</div>
<div class="mcetbyclass">
```

```

        <h1>element1</h1>
        <p>element2<p>
        <h3>element3</h3>
    </div>
    <div class="mcetbyclass">

        <h1>element1</h1>
        <p>element2<p>
        <h3>element3</h3>
    </div>
    <script>
        var v=document.getElementsByTagName("h1")
        console.log(v)
    </script>
<!--
4.document.querySelector(): Returns the first element that matches the specified selectors
-->
    <div class="mcetbyclass">

        <h1>element1</h1>
        <p>element2<p>
        <h3>element3</h3>
    </div>
    <div class="mcetbyclass">

        <h1>element1</h1>
        <p>element2<p>
        <h3>element3</h3>
    </div>
    <script>
        var vari=document.querySelector(".mcetbyclass")
        console.log(vari)
    </script>
<!--
5.document.querySelectorAll(): Returns the all elements that matches the specified selectors
-->
    <div class="mcetbyclass">

        <h1>element1</h1>
        <p>element2<p>
        <h3>element3</h3>
    </div>
    <div class="mcetbyclass">

        <h1>element1</h1>
        <p>element2<p>
        <h3>element3</h3>
    </div>
    <script>
        var vari1=document.querySelectorAll(".mcetbyclass")
        console.log(vari1)
    </script>

<!-- properties to select html elements
document.body
document.head
document.title
document.anchors
document.images
document.forms
document.scripts
-->

<script>
    console.log(document.scripts)
</script>

<!-- CREATING A HTML ELEMENT -->
<div id="parent">
    <p id="trial1">thi is example of creating element using html</p>
    <p id="trial2">thi is example of creating element using html</p>
</div>
<script>
    var element = document.createElement('p');

    // <!-- SET CONTENT TO ELEMENT -->
    element.innerText="this is created by js"
    console.log(element)

    // <!-- APPEND A NEW ELEMENT -->

```

```

var parent=document.getElementById("parent")
console.log(parent)
parent.appendChild(element)
console.log(parent.innerText)
console.log(parent.innerHTML)

// <!-- INSERTING A ELEMENT BEFORE ELEMENT -->
var trial=document.getElementById("trial2")
parent.insertBefore(element, trial)

// <!-- REMOVING AN ELEMENT -->
var remove=document.getElementById("trial1")
remove.remove()

// <!-- REMOVE CHILD ELEMENT -->
var rem=document.getElementById("trial2")
parent.removeChild(rem)

// <!-- REPLACE AN ELEMENT -->
var element1 = document.createElement('p');
element1.innerText="this is created by js second time"
element.replaceChildren(element1)
</script>

<!-- JavaScript Style Methods -->
<script>
// Select an element to style
var styledElement = document.getElementById("mcetbyid");

// Change color using JavaScript
styledElement.style.color = "blue";

// Change font size using JavaScript
styledElement.style.fontSize = "24px";

// Add a background color using JavaScript
styledElement.style.backgroundColor = "lightgray";

// Adding multiple styles at once
Object.assign(styledElement.style, {
  border: "2px solid black",
  padding: "10px",
  textAlign: "center"
});

console.log(styledElement);
</script>

<!-- Another example with class elements -->
<script>
// Select multiple elements by class
var classElements = document.getElementsByClassName("mcetbyclass");

// Loop through all selected elements and apply styles
for (var i = 0; i < classElements.length; i++) {
  classElements[i].style.border = "1px solid red";
  classElements[i].style.padding = "5px";
  classElements[i].style.margin = "5px";
}

console.log(classElements);
</script>

<!-- JavaScript Style Methods - Advanced Example -->
<script>
// Select elements to apply styles
var container = document.getElementById('parent');
var element = document.getElementById('mcetbyid');

// Apply general container styles
container.style.display = "flex";
container.style.flexDirection = "column";
container.style.alignItems = "center";
container.style.justifyContent = "space-between";
container.style.padding = "20px";
container.style.border = "2px solid #ccc";
container.style.borderRadius = "10px";
container.style.backgroundColor = "#f9f9f9";
container.style.width = "80%";
container.style.margin = "0 auto";

// Apply text styles to an element

```

```

element.style.color = "#2c3e50";
element.style.fontFamily = "'Arial', sans-serif";
element.style.fontSize = "28px";
element.style.fontWeight = "bold";
element.style.textAlign = "center";
element.style.textTransform = "uppercase";
element.style.textShadow = "2px 2px 5px rgba(0,0,0,0.3)";

// Add margin and padding
element.style.margin = "10px 0";
element.style.padding = "15px";

// Apply border and background styles
element.style.border = "2px solid #2980b9";
element.style.borderRadius = "5px";
element.style.backgroundColor = "#ecf0f1";

// Adding box-shadow and transform for a 3D effect
element.style.boxShadow = "0 4px 8px rgba(0,0,0,0.2)";
element.style.transform = "scale(1.05)";
element.style.transition = "transform 0.3s ease-in-out";

// Hover effect using JavaScript event
element.addEventListener('mouseenter', function() {
    element.style.transform = "scale(1.1)";
    element.style.boxShadow = "0 6px 12px rgba(0,0,0,0.3)";
});

element.addEventListener('mouseleave', function() {
    element.style.transform = "scale(1.05)";
    element.style.boxShadow = "0 4px 8px rgba(0,0,0,0.2)";
});

// Working with pseudo-elements
var pseudoStyle = document.createElement('style');
pseudoStyle.textContent = `
    #mcetbyid::after {
        content: " - Styled by JS";
        color: #8e44ad;
        font-size: 14px;
        display: block;
        margin-top: 10px;
    }
`;
document.head.appendChild(pseudoStyle);

// Responsive design using media queries
var mediaStyle = document.createElement('style');
mediaStyle.textContent = `
    @media (max-width: 768px) {
        #parent {
            width: 100%;
            padding: 10px;
        }
        #mcetbyid {
            font-size: 22px;
        }
    }
`;
document.head.appendChild(mediaStyle);
</script>

</body>
</html>

</body>
</html>

```

7. Event Handling

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

  <!-- inline event handlers(using event attributes in HTML)-->
  <button id="btn" >submit</button>
  <input type="text" id="input">
</body>
<script>
/*
  EVENT HANDLING

  EVENT
  :- event is nothing but an action
  :- in webpage also what ever user do everything is an event

  WAYS OF GENERATING AN EVENT
  :- user-> keypress, scroll, focus..
  :- system-> load, error, abort..

  TYPES OF USER GENERATED EVENTS
  :- browser specific events -> scroll up/down resize browser
  :- DOM/webn page specific events ->click, hover, focus..

  Event handler is function we write to run when an event happens
  */

/*
  COMMONLY USED EVENTS IN JAVASCRIPT

  Mouse events
    click
    doubleclick
    mouseover
    mouseout
    mouse move

  Keyborad events
    key down
    key up
    key press
  Focus events
    focus
    blur
    focusin
    focusout
  Form events:- only used for forms
    submit
    reset
    change
  */
/*  WAYS TO HANDLE EVENTS
  1.inline
  2.internal
  3.using event properties in java script
  4.using addEventListener() in java script

  */
//INTERNAL
//  function change(){
//    document.body.style.backgroundColor='grey'
//  }

  let v1=document.getElementById("btn   gt=''")
  let v2=document.getElementById("input")
v1.onclick=change
function change(){
  document.body.style.backgroundColor='grey'
}
// v2.addEventListener('focus',change)
// function change(){
//   console.log(event)
//   v2.style.backgroundColor='green'
// }

</script>
</html>
```

End of organized trainer notes